FVL

A declarative specification layer for financial coordination systems

Akande Mayowa

0. PREFACE

The last decade of decentralized development has been a well documented exercise in perfecting the infrastructure. We have moved from the foundational breakthrough of Nakamoto's consensus to the sophisticated scaling of L2 architectures. For the first time, we have the infrastructure to support compute on a global scale. We have built the faster chains we were promised; the industry has successfully solved for capacity.

Yet, despite this maturity, a fundamental contradiction remains. We have reached a point where audits and formal verification are standard, yet security in the space remains a perpetual game of *"patching the wrong things."* We treat smart contract exploits as edge cases or developer errors, but they are the predictable outcomes of what our execution environment allows. We are building robust financial vaults on top of foundations that are, by design, too permissive. We have mistaken unrestricted programmability for financial utility.

When we take a closer look, every financial system ever deployed, from a simple treasury to a complex staking mechanism, is a composition of the same finite set: who can participate, what moves, under what conditions, by what formula, and across what time horizon.

There is a small, legitimate class of protocols, Aave and Compound being the primary examples, where the complexity is real and the need for general-purpose computation of a Turing-complete EVM serves the necessary purpose. But they are the exception, not the rule. The vast majority of financial coordination does not require a general-purpose execution layer.

If a system can be fully described by its primitives, then a wrong state transition should not merely be unlikely, it should be structurally impossible. The industry has spent years building a chain for everything. The logical conclusion of this evolution is not more speed or lower fees, but a specialized coordination layer designed for this exactly: an environment where the logic of finance is encoded into the architecture itself.

TABLE OF CONTENTS

0. PREFACE
1. NATURE OF FINANCIAL COMPUTATION
2. THE ARCHITECTURE OF COORDINATION
3. THE EXECUTION MODEL
4. THE ROLLUP ARCHITECTURE
5. FVL IMPROVEMENT PROPOSALS (FIPs)
6. CONCLUSION
7. FURTHER READING

1. NATURE OF FINANCIAL COMPUTATION

Finance has always been a bounded rule set. From the earliest grain loans to modern clearinghouses, every financial system ever constructed has been a constraint system at its core: a defined set of participants, defined asset(s), a defined trigger, a defined formula, a defined time horizon. Nothing outside those bounds is permitted or relevant. This has never changed. What changed is the execution environment we chose to build them on.

At its most fundamental level, every financial system is a composition of five primitives:

- *pool: Asset collection rules*

- *rules: Conditional logic and distributions*

- *rights: Permission definitions*

- *time: Temporal constraints*

- *oracles: External data feeds*

The combination of these primitives yield the following:

i. *Lending Pool: pool + rights + oracles + rules*
ii. *Staking Pool: pool + time + rights*
iii. *Crowdfunding: pool + time + rules + rights*
iv. *DAO: pool + rights + rules + oracles + time*

The variation is in the parameters, not in the primitive set itself. As long as a system can be described end-to-end, these five primitives are sufficient to construct it.

Solidity was not designed for this. It was designed to solve a legitimate problem: Bitcoin's scripting language was far too limited, too restrictive, incapable of expressing complex programmable logic. Turing-completeness was the answer, and it was the right answer for that problem. It made truly novel protocols possible and placed decentralized finance at the forefront of financial innovation. This is not in dispute. The question is more specific: does every use case require Turing-completeness? The answer is no, and financial systems are the clearest example of that. With some exceptions, where the complexity of interest rate models and liquidation logic genuinely requires general computation, the vast majority of financial systems do not need it. Financial systems at their fundamental level are described, not programmed. They require deterministic, auditable, and secure operations.

In 2016, the DAO was drained for 3.6 million ETH through a reentrancy vulnerability. The root cause was not a developer error in the conventional sense. It was the predictable outcome of deploying a constrained financial system on an unconstrained execution layer:

```
contract VulnerableDAO {

 mapping(address => uint256) public balances;

function deposit() external payable {
  balances[msg.sender] += msg.value;
}

function withdraw(uint256 _amount) external {
  require(balances[msg.sender] >= _amount, "Insufficient balance");
  (bool sent, ) = msg.sender.call{value: _amount}("");
  require(sent, "Failed to send Ether");
  balances[msg.sender] -= _amount;  // state updated after external call
}

}
```

The withdraw function sent ETH before updating the balance, and because the EVM permits reentrant calls, an attacker could recursively call withdraw before the balance was decremented. The FVL equivalent of the same contract:

```
system: "DAO"

pool:

 collect:

  from:

   type: anyone

  what:

   type: eth

  min:

   type: zero

  max:

   type: none

  cap:

   type: none
```

```yaml
rules:
  conditions:
    - if:
        type: balance_gte
        value: "1"
      then:
        type: enable
        permission: withdraw
  distribute:
    formula:
      type: proportional
    to:
      type: contributors
    triggers: manual

rights:
  anyone: [deposit, view]
  contributors: [withdraw]

time:
  start:
    type: now
  end:
    type: none
  locks:
    type: none
```

```
vesting:
  type: none


oracles: []
```

In FVL that class of bug is entirely erased, not by adding a reentrancy guard, not by reordering logic, but because the execution environment does not permit that state transition to exist in the first place.

2. THE ARCHITECTURE OF COORDINATION

Financial coordination is fundamentally about deterministic agreements. Not programs, not general computation, but agreements, participants, assets, conditions, outcomes. FVL enforces these agreements by treating every financial system as a finite state machine rather than an open-ended program, and by abstracting all financial logic into five primitives that together cover the complete lifecycle of value. FVL deliberately limits expressiveness to ensure safety and determinism.

Loops in the context of a financial system have been shown in the past to enable unwanted and malicious behaviors, such behavior include griefing, for a closed system, loops are unnecessary and just bring about complication in operations. The use of finite primitives intentionally limits the expressiveness and that allows for a bounded attack surface, an expressive primitive set creates an unbounded attack surface and with the combination of existing capabilities such as loops and recursions, can be used to produce sophisticated attack vectors.

The implication of these constraints is deterministic execution. Given the same initial state and the same transaction sequence, an FVL system always produces an identical financial state.

The Primitive Set

Every FVL system is composed of five functional sections that together define the complete lifecycle of value. The full skeleton of a system is as follows:

```
system: <string>        # System name
pool:   <pool_config>   # Asset collection rules
rules:  <rules_config>  # Conditional logic and distributions
rights: <rights_map>    # Permission definitions
time:   <time_config>   # Temporal constraints
oracles: <oracle_list>  # External data feeds
```

Pool:

is the entry point of the system. It defines asset collection rules: who can contribute, whether that is anyone, a whitelist, or defined token holders, and which assets are accepted. It enforces structural safety through minimum, maximum, and total cap requirements, preventing unexpected behaviors at the boundary conditions that have historically been the source of exploits, here only authorized tokens and users will be allowed to partake in a system, this enforces tight enforcement over what is accepted and who can participate.

Rules:

encode declarative conditional logic. If-then statements trigger state changes, enabling logic only when defined conditions have been met. Distribution formulas govern how value flows out of the system.

Rights:

Rather than implicit authority, FVL requires explicit mapping of roles to permissions. Administrative powers such as pausing the system, and user actions such as claiming rewards, are correctly preassigned, this allows for a proper and explicit mapping of who can do what, and enforces role based access control.

Time:

Financial activity is time-bound. This primitive manages start and end constraints, lock-up periods, and vesting schedules, whether linear, cliff-based, or milestone-driven.

Oracles:

Financial coordination frequently depends on data from outside the system, asset prices being the most common example. This section defines the named data feeds the system is permitted to consume, ensuring that every external dependency is declared, audited, and explicit rather than dynamically resolved at runtime.

In the domain of financial coordination, Pool, Rules, Rights, Time, and Oracles cover any financial instrument that can be described end-to-end. Every treasury, staking mechanism, lending pool, vesting contract, crowdfunding system, and DAO ever deployed on Ethereum is a composition of this set. FVL does not claim to be sufficient for the construction of every financial system that exists and will exist. There are protocols whose complexity genuinely requires general computation, and warrant such usage, and are outside its intended scope and are not better served by it. This is an intentional boundary. The primitive set is deliberately scoped to the class of systems where safety and determinism matter more than expressiveness, which is the vast majority of financial coordination systems that have ever been deployed.

The primitive set is not permanently fixed. An improvement protocol governs the addition of new primitives, with a single constraint: any proposed primitive must satisfy the non-Turing-completeness requirement and must be deemed necessary to express financial systems that cannot otherwise be described.

Each FVL system is identified by a unique SystemID, derived through Content-Addressed Identity. The SystemID is a deterministic *Keccak-256* fingerprint of the system's canonical JSON representation. Two systems with identical definitions will always produce the same SystemID. Any change to the definition, any field, any condition, any permission, produces a different ID. Because the SystemID is derived from content rather than from deployment context, identical templates are deduplicated at the protocol level. Deploying the same template twice is rejected. There is one canonical instance of any given system definition on-chain. And any party can verify that a deployed system matches an expected definition by hashing the template locally and comparing.

SystemIDs are also primitive for composition. A deployed FVL system can reference another deployed system by its SystemID, enabling systems to be built on top of other systems. A yield aggregator can route deposits into a lending pool. A treasury can delegate distributions to a vesting system, composition of this kind does not require either system to be aware of the other at deploy time. The referencing system declares a dependency by SystemID, and the runtime resolves it against the set of deployed systems at execution time.

A single constraint governs how composition works, circular references are prohibited. System A cannot reference System B if System B references System A, directly or transitively. This constraint exists because circular dependencies would make state transitions non-deterministic in their ordering. If A's execution depends on B's state and B's execution depends on A's state, the correct evaluation order is undefined. The no-circular-reference rule eliminates this

ambiguity entirely by enforcing a strict dependency graph with a clear topological order. In practice, this means compositions always flow in one direction, higher-order systems consume the outputs of lower-order systems, and the lower-order systems have no knowledge of what depends on them.

3. THE EXECUTION MODEL

Every FVL system is a finite state machine. This is the foundation on which FVL's security and verifiability properties rest. General-purpose smart contract platforms are Turing-complete. They can express arbitrary computation, which means they can also express arbitrary failure modes. Reentrancy attacks, unbounded loops, integer overflows, and state corruption under adversarial input are all consequences of unbounded expressiveness.

Systems as State Machines

A finite state machine is defined by five components: a finite set of states $\boldsymbol{S}$, an input alphabet $\boldsymbol{\Sigma}$, a transition function $\delta : S \times \Sigma \to S$, an initial state $\boldsymbol{S_0}$, and a set of terminal states $\boldsymbol{F}$.

In FVL, $\boldsymbol{S}$ is the set of all possible configurations of the global state, covering every combination of balance values, oracle readings, deployed systems, and account nonces. This set is astronomically large in practice but formally finite, bounded by the numeric limits of the types that represent it. $\boldsymbol{\Sigma}$ is the set of all valid transactions, typed and structured inputs drawn from a finite alphabet. $\boldsymbol{\delta}$ is the runtime's transition function: a pure function that takes a state and a transaction and deterministically produces the next state. Every transition is atomic, it either commits fully or leaves the state unchanged. $\boldsymbol{S_0}$ is the genesis state: empty balances, no deployed systems. $\boldsymbol{F}$ is not fixed globally; each deployed system defines its own terminal conditions as predicates over the state, expressed as rules within the system definition.

Content-Addressed Identity

Each deployed system is identified by a System ID:

$$\text{system_id} = \text{Keccak256}(\text{canonical_json}(\text{system}))$$

The System ID is a deterministic fingerprint of a deployed system's logic. Any change to the definition, any field, any condition, any permission, produces a different ID. Three properties follow directly: the on-chain binding of system ID to system logic is permanent and cannot be modified under an existing ID; anyone can verify locally that a deployed system matches the definition they expect without trusting the deployer; and duplicate deployment is rejected at the protocol level.

Deploying and Interacting

When a system is deployed, the template is parsed, validated, and hashed to produce its System ID. The resulting deploy transaction is applied atomically: the system is registered in state, oracle slots declared in the template are initialized, and deployer metadata is recorded. The template itself is persisted separately so it remains available for state reconstruction, the block log records only the System ID.

When a user interacts with a deployed system, the runtime evaluates the system's conditions against the current state in declaration order. Each condition tests a predicate over oracle values, balances, or time and if satisfied, applies the corresponding action immediately. State mutations from earlier conditions are visible to later ones within the same transaction. Because neither conditions nor actions can invoke arbitrary code or loop unboundedly, the complete set of state changes any interaction transaction can produce, is fully determined by reading the template.

Every transaction carries a monotonically increasing nonce tied to the sender. A transaction whose nonce does not exactly match the sender's current counter is rejected before any other validation occurs. This ensures every transaction is valid at most once and executes in the order it was submitted.

State Roots and Verification

Every successful transaction produces a new state root:

$$\text{state_root} = \text{Keccak256}(\text{canonical_bytes}(\text{state}))$$

Computing canonical bytes requires sorting all state maps by their keys before serialization. Without an explicit sort, hash map iteration order is non-deterministic, two nodes processing identical transactions could produce different roots. The sort step is what makes state roots deterministic across independent parties.

State roots are embedded in each block and anchored to Ethereum's mainnet. Because the transition function is public and the block log is published as data availability, any party can independently replay the sequence and verify a submitted root without trusting the sequencer. A challenger disputing a root needs only the log and the system registry. This is the concrete mechanism behind the fraud-proof guarantee.

What the Finite State Machine (FSM) Guarantee Provides:

Determinism:

The transition function reads no external state during execution, no timestamps, no randomness, no oracle calls mid-transition. Given the same initial state and the same ordered transaction sequence, any two parties arrive at the same result,this is the precondition for everything else, this ensures that state transitions are completely bounded and no interference is possible mid execution.

Termination:

Every transaction terminates in *O(k)* time, where k is the number of conditions defined in the system at deployment, k is fixed at deployment and cannot change, hence no transaction can cause k to grow from its deployed state. Execution costs are perfectly predictable, and an entire class of denial-of-service vectors does not exist.

Replayability:

Because the transition function is deterministic and all inputs are recorded in an append-only log, the current state can always be reconstructed from scratch. Any party with access to the log can verify independently that the reported state is indeed correct.

Independent Verification:

Anyone can verify a state root claim by replaying the relevant transactions. Because FVL systems are finite state machines with a public, deterministic transition function, any invalid state transition is provably wrong by construction. A challenger needs only the block log and the system registry to produce a cryptographic proof that a submitted root is incorrect. This property is structurally simpler to realize in FVL than in a general-purpose rollups, there is no arbitrary bytecode to re-execute, only a fixed set of typed conditions and actions evaluated against a known state.

Bounded attack surface:

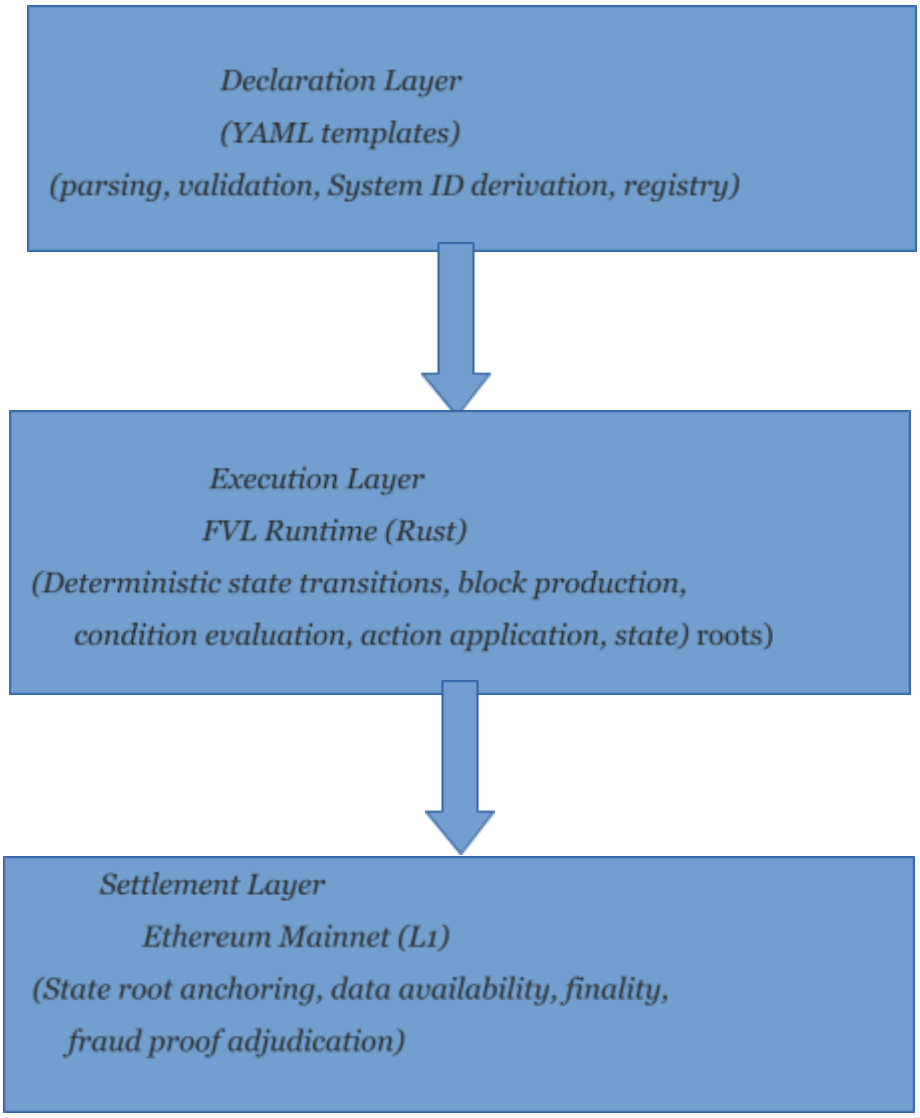
The input alphabet is typed and finite. It is not possible to submit a transaction that calls an arbitrary function, references an undeclared oracle, or triggers behavior outside the defined primitive set.

4. THE ROLLUP ARCHITECTURE

FVL utilizes the Optimistic Rollup design stack that existing rollups operate on. It inherits Ethereum's security guarantees as its settlement layer while replacing the EVM execution environment with a purpose-built, constrained runtime for financial systems. This section describes the three-layer architecture precisely, explains how state is anchored to Ethereum mainnet, and details how the optimistic model operates within FVL's design.

THE ARCHITECTURAL MODEL

FVL's architecture is organized into three distinct layers, each with a clearly defined responsibility.



Declaration Layer

Users write YAML templates that describe pools, rules, conditions, actions, time constraints, and access controls using FVL's primitive set. The parser deserializes each template into a typed system struct, the validator then checks for structural correctness, and the System ID is derived as a content-addressed hash of the parsed definition. This layer has no on-chain presence, it runs entirely off-chain, before any transaction is submitted, this serves as the entry point for every FVL system's creation.

Execution Layer

This is the FVL runtime, a Rust implementation of a deterministic finite state machine that processes transactions and produces new states. It does not implement the EVM. It does not execute bytecode, does not support arbitrary computation, and does not carry a gas metering model based on opcode costs. Instead, it evaluates a fixed set of typed primitives against a structured state, producing state transitions that are bounded in time and fully predictable in cost. The sequencer operates at this layer, it receives FVL transactions, maintains their ordering, applies them to the current state via the transition function, produces blocks, and publishes the block log, each block includes a state root, a cryptographic commitment to the full state after that block's transactions have been applied. The execution layer is the only structural replacement FVL makes to the standard Optimistic Rollup stack. The block log, the state root commitment scheme, the settlement contract, the data availability model, and the fraud proof mechanism all follow established optimistic rollup design.

Settlement Layer

The Ethereum mainnet is utilized as the settlement layer, its role is to serve as the anchor: it holds the state root commitment contract, provides data availability guarantees for the block log, and adjudicates fraud proof challenges. It does not re-execute FVL transactions. It only verifies that state roots submitted by the sequencer are consistent with the published transaction data.

Rollup components

FVL's rollup architecture comprises the same components found in established Optimistic Rollup designs.

The sequencer batches FVL transactions and submits them to the block log. Transaction data is posted to Ethereum as calldata, providing data availability. The state root commitment contract on L1 records the cryptographic commitment to off-chain state at each block. A challenge contract adjudicates fraud proof disputes during the challenge window. A bridge contract handles deposits from L1 to L2 and processes withdrawal requests from L2 back to L1. The sequencer can operate under a single operator or be distributed across multiple operators under a leader election protocol.

State root computation and anchoring

After each block is produced, the sequencer computes a state root:

$$\text{state_root} = \text{Keccak256}(\text{canonical_bytes}(\text{state}))$$

Canonical bytes are produced by sorting all four state maps, systems, balances, oracles, and nonces, by their keys, then serializing the sorted structure to JSON. Sorting is not optional: hash map iteration order is non-deterministic, and without a canonical ordering, two independent parties replaying the same transactions could compute different roots from the same underlying state. The sort step guarantees that the state root is a function only of the state's contents, not of the order in which entries happen to sit in memory.

State roots are embedded in each block. The submitter process watches the block log and calls *submitStateRoot(block_number, state_root)* on the FvlRollup contract deployed to Ethereum, anchoring the off-chain state to L1. Once a state root is on-chain it is permanent and publicly verifiable.

Anyone wishing to verify a state root independently needs two things: the block log, which contains all transactions in order, and the system registry, which contains the YAML definitions for all deployed systems. With both, they can replay every transaction from genesis, compute the state root at any block height, and compare it to what the sequencer submitted on-chain.

The Sequencer

The sequencer is the entity responsible for ordering FVL transactions, applying them to the current state, producing blocks, and publishing the block log. It is the only entity authorized to submit state roots to the settlement contract.

In a centralized configuration, users extend trust to the sequencer operator. The operator can theoretically omit transactions or submit an incorrect state root, and the system's response to either is the fraud proof mechanism described below. The centralized model is operationally simple and sufficient for environments where the operator is known and trusted.

In a decentralized configuration, block production is distributed across multiple operators under a leader election protocol. No single operator controls ordering permanently. Operators in this model post bonds that are subject to slashing if a submitted state root is successfully challenged, replacing operator trust with an economic security assumption. The settlement contract and fraud proof system are indifferent to which configuration is in use: they adjudicate outputs, not operators.

Fraud proofs and dispute settlements

An Optimistic Rollup accepts state roots as valid by default. Verification occurs only when a challenger disputes a submission. The challenge window, the period during which a fraud proof can be submitted, is what gives the system its security guarantee. A state root that survives the challenge window without a successful dispute becomes final.

The fraud proof mechanism works as follows.

i. Challenge submission. A challenger who believes a submitted state root is incorrect identifies the specific block in question and submits a challenge to the FvlRollup contract on L1, along with a bond

ii. Bisection. An interactive bisection protocol narrows the dispute to a single transaction, the precise state transition the challenger claims is wrong. This reduces the L1 verification cost from replaying an entire batch to verifying a single step.

iii. Single-step verification. The L1 contract verifies the disputed state transition by applying the transaction to the pre-dispute state and checking whether the result matches the submitted post-state root. In a general-purpose EVM rollup, this requires an on-chain EVM interpreter, a component of significant complexity. In FVL, it requires only the FVL transition function applied to one transaction. Because the transition function is a fixed, finite set of typed operations over a structured state, the on-chain verifier is dramatically simpler. Every valid state transition is deterministic and fully specified by the system's declared rules. An invalid transition is unambiguous: the verifier computes what the output should be and compares it to what was submitted.

iv. Resolution. If the on-chain verification confirms the challenger's claim, the incorrect state root is removed. In a decentralized sequencer configuration, the offending operator's bond is slashed and the challenger receives a reward. If the submitted root is correct, the challenge fails and the challenger's bond is slashed.

v. Challenge period. During the challenge window, withdrawals and cross-chain operations that depend on a given state root are locked. Once the window closes without a successful challenge, the state root is considered final and downstream operations proceed.

The security of any Optimistic Rollup's fraud proof system ultimately rests on the correctness of its on-chain verifier. In a general-purpose rollup, that verifier must correctly implement the EVM. In FVL, the challenge contract is a Solidity implementation of the FVL transition function: a fixed, enumerable set of typed operations over a structured state. The trusted computing base is small enough to audit completely. This is a direct consequence of the constrained execution model, and it is one of the strongest properties of the architecture.

5. FVL IMPROVEMENT PROPOSALS (FIPs)

FVL's primitive set is intentionally finite. This is the source of its safety guarantees, and it begs the question: what happens when the primitives are not enough? The answer is a formal extension process, and the design of that process matters as much as the primitives themselves.

The current primitive set covers the patterns that appear repeatedly across financial coordination systems: access control by token or identity, asset collection with min and cap constraints, conditional logic over balances, prices, time, and holder counts, certain distribution formulas, vesting schedules, and a rights model for role-based permissions. These are not arbitrary choices. They are the smallest set that can express staking systems, lending pools, treasuries, crowdfunding campaigns, and yield aggregators without requiring any general computation.

Every primitive in the set has a property that is essential for the system's security model: it is statically analyzable. Given a template, it is possible to determine the complete set of state transitions the system can produce, without executing it. This property does not survive the introduction of arbitrary computation. The moment a primitive requires a loop or recursion, static analysis becomes undecidable and the fraud proof verifier becomes arbitrarily complex.

FVL Improvement proposals

New primitives are added through FVL Improvement Proposals, or FIPs. The FIP process is modeled on Ethereum's EIP process, adapted for FVL's domain-specific constraints.

A FIP must specify the following.

i. **Use case.**

A concrete financial coordination pattern that cannot be expressed using any combination of existing primitives. Proposals that can be reduced to existing primitives through composition are rejected at this stage. The burden of proof is on the proposer to demonstrate that the gap is real.

ii. **Proposed syntax**.

The YAML syntax for the new primitive, consistent with FVL's existing type system and naming conventions. The primitive must fit naturally into one of FVL's five top-level sections: pool, rules, rights, time, oracles. Proposals that require new top-level sections or changes to the type system carry a higher acceptance threshold. The proposer must also define the Canonical JSON mapping used for System ID derivation:

$$\text{system_id} = \text{Keccak256}(\text{canonical_json}(\text{system}))$$

iii. **Static analysis proof.**

A demonstration that the primitive's evaluation terminates in bounded time O(1) and that its complete behavior can be determined without execution. This is the non-negotiable technical requirement. Any primitive whose behavior depends on unbounded iteration, recursion, or runtime-variable branching, fails this criterion unconditionally.

iv. **Security analysis.**

An assessment of the attack surface introduced by the new primitive. This includes: whether the primitive can be combined with existing primitives to produce behavior that violates the system's safety invariants, whether it introduces new oracle dependencies, and whether it changes the complexity of the fraud proof verifier.

v. **Reference implementation**.

A Rust implementation of the primitive in the FVL runtime, and a corresponding Solidity implementation suitable for inclusion in the on-chain challenge contract. Both implementations must produce identical outputs for all valid inputs. The Solidity implementation is a first-class deliverable because it determines the trusted computing base of the fraud proof system.

The acceptance criterion

The acceptance criterion for a FIP is a single question: can this primitive be implemented without Turing-completeness?

If the answer is yes, the primitive is a candidate for inclusion. It must still clear the security analysis and the reference implementation requirements, but it is inside the design space of FVL

If the answer is no, the use case belongs on Ethereum proper. This is not a judgment about the use case's validity. Complex financial logic that requires general computation has a home: Solidity, Vyper, or any other EVM-compatible language. FVL is not a replacement for that stack. It is a specialized layer for the class of financial systems whose logic can be fully expressed as finite state transitions over typed primitives. When a proposed use case falls outside that class, the correct answer is to build it on Ethereum directly, not to expand FVL's execution model to accommodate it.

This boundary is load-bearing. A version of FVL that adds general computation to accommodate edge cases is a version of FVL that loses its static analysis properties, its simple fraud proof verifier, its guarantee of bounded execution, and ultimately its claim to be safer than writing Solidity. The acceptance criterion exists to prevent that erosion.

Primitives categories and extension surface

The current primitive set is organized across five categories. Each category has a different extension surface and different constraints on what new primitives can look like.

i. Access rules govern who can interact with a system. The current set covers open access, token-gated access, NFT-gated access, explicit whitelists, and minimum balance requirements. New access rules are the most straightforward category to extend. They reduce to a predicate over address properties, which is inherently bounded.

ii. Expressions are the condition half of the if-then rules that drive system behavior. The current set covers balance comparisons, price conditions via oracle, time conditions, holder counts, total value tracking, collateral ratios, utilization rates, and named events. Extensions here require careful analysis because expressions compose: a new expression type can be combined with any existing action, so its introduction multiplies across the entire condition-action space. Proposed expressions must be evaluable against a single snapshot of state without requiring iteration over a variable-length set *O(1)*.

iii. Actions are the then half of condition rules. The current set covers enabling and disabling permissions, liquidation, minting, burning, transferring, and pausing.

Extensions here directly affect what state changes a system can produce, and therefore directly affect the fraud proof verifier. Every new action type requires a corresponding implementation in the on-chain challenge contract. This is the highest-cost category to extend.

iv. Distribution formulas govern how value flows to recipients. The current set covers proportional, equal, weighted, tiered, and quadratic distributions. These are all closed-form computations over a known set of contributors. Extension candidates, such as time-weighted distributions or performance-adjusted allocations, must remain closed-form. A distribution formula that requires iterating over a variable-length history *O(n)* is not compatible with FVL's *O(k)* execution model.

v. Time mechanics govern when things happen. The current set covers locks, linear vesting, cliff vesting, graded vesting, and milestone-based vesting. Extensions here are generally low-risk since time mechanics reduce to comparisons against timestamps and durations, which are inherently bounded.

What the FIP process protects against

The FIP process exists to keep two properties intact as the primitive set grows.

The first is auditability. A system built from FVL primitives can be audited by reading its template. This property depends on the primitive set being small enough that a reader can hold the complete behavioral space in their head. Each new primitive expands that space. The FIP process ensures that each expansion is deliberate, documented, and justified by a real use case rather than speculative coverage.

The second is verifier simplicity. Every primitive in FVL's execution model must be implementable in the on-chain challenge contract. The complexity of that contract is the complexity of the fraud proof system's trusted computing base. A FIP that introduces a primitive whose Solidity implementation is fifty lines is a different kind of change than one whose implementation is five hundred. The reference implementation requirement makes this cost visible at proposal time rather than at deployment time.

6. CONCLUSION

Financial systems are not arbitrary programs. They are finite compositions of a small set of primitives, and the industry has been using a general-purpose execution environment to express them because no better layer existed. We have paid for this mistake with billions lost in exploits. Exploits that were not always edge cases or developer errors, they were the predictable output of an execution environment with no financial invariants, no logic boundaries, and no structural constraint on what a state transition is allowed to do.

The infrastructure is mature. We have the throughput, the settlement guarantees, and the tooling. What the ecosystem lacks is a layer where financial logic is correct by construction rather than correct by careful implementation. The gap between general-purpose code and financial safety is a structural flaw, and not always a developer skill issue. Security in DeFi has been treated as something you negotiate for, from an audit firm after the code is written. The execution environment was never designed to enforce financial invariants by default, and no amount of auditing changes what the architecture permits.

A financial system declared in FVL will behave as declared. The rest of the stack, the throughput, the composability, the settlement, is a solved problem. We are standing on a decade of battle-tested infrastructure. What was missing was a layer where the logic of financial coordination is the architecture, not an application running on top of it.

7. FURTHER READING

Protocols

Aave Documentation https://aave.com/docs

Compound Documentation https://docs.compound.finance

Languages

Vyper Language https://vyperlang.org

Smart Contract Languages — curated resource list covering non-Turing-complete alternatives https://github.com/s-tikhomirov/smart-contract-languages

Foundational Papers

Ethereum Whitepaper — Vitalik Buterin (2014) https://ethereum.org/whitepaper

Ethereum Yellow Paper — Gavin Wood https://ethereum.github.io/yellowpaper/paper.pdf

Do Smart Contract Languages Need to be Turing Complete? (2019) https://www.researchgate.net/publication/332072371_Do_Smart_Contract_Languages_Need_to_be_Turing_Complete

Exploits

The DAO Hack https://medium.com/swlh/the-story-of-the-dao-its-history-and-consequences-71e6a8a551ee

Rekt Leaderboard https://rekt.news/leaderboard

Infrastructure

Optimistic Rollups https://ethereum.org/developers/docs/scaling/optimistic-rollups

Optimism OP Stack Documentation https://docs.optimism.io

OP Stack Technical Specification https://specs.optimism.io